A Mini Project Report of

PCC-201-COM : Data Structures

"Implementation of Keyword Frequency Counter"

Submitted by

JADHAV SHRIPAD SURESH (39)

Under the Guidance of

Prof. Tejal. H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S

COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "Mini Project Report" submitted by **JADHAV SHRIPAD SURESH (39)**, on **"Implementation of Keyword Frequency Counter"** is work done by him and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of engineering in computer Engineering, at Bharati Vidyapeeth's College of Engineering, Lavale, Pune.

Prof. Tejal. H. Patil
(Guide)

Dr. Lalit. V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

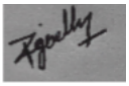
It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to Prof. Tejal. H. Patil Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to Dr. Lalit. V. Patil, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to Prof. Dr. R. N. Patil Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

JADHAV SHRIPAD SURESH

Student Name



ABSTRACT

The Keyword Frequency Counter is a Python-based mini project developed to demonstrate the practical application of Hashing, Collision Resolution, and Sorting algorithms in Data Structures. The system processes text paragraphs, documents, or source code files to manage lexical information such as unique keywords, word lengths, hash bucket indices, occurrence counts, and relative percentage share.

The application allows users to input or load text paragraphs, normalize tokens by removing punctuation and common English stop words, search for specific keywords using their hash keys, sort keywords according to their occurrence frequency, and identify the Top-5 most dominant terms in a document.

A customized Hash Table with Separate Chaining (using a Polynomial Rolling Hash function with prime modulus 101) is used to index and update word frequencies in O(1) average time, while Bubble Sort is used to arrange keyword records in descending order of frequency for rapid text summarization and lexical analysis.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List Of Figures	iv
1	Introduction a) Overview b) Problem Statement c) Objective	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) Existing system b) Proposed system c) Working Diagram d) System code	5
6	Result	8
7	Application	9
8	Future Scope	10
9	Conclusion	11
10	References	12

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Case Diagram	03
Fig 5.1	System Design (ER Diagram)	06
Fig 5.2	System Code	07
Fig 5.3	Result	08

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

1. INTRODUCTION

1.1 Overview

The Keyword Frequency Counter is a Python-based mini project designed to process text documents, count word occurrences, and demonstrate the practical implementation of Hashing and Sorting algorithms in Data Structures.

In modern information retrieval systems, search engines, and text analytics platforms, users need to scan large paragraphs of text, filter out common filler words, check how many times specific keywords appear, and identify the most dominant terms. Managing and counting words manually or in unsorted lists becomes time-consuming as document size grows.

The proposed Keyword Frequency Counter provides a simple, menu-driven application that allows users to input text, view indexed keywords, search for specific word counts in O(1) average time using a Hash Table, and sort keywords according to their frequency using Bubble Sort.

1.2 Problem Statement

To design and implement a menu-driven Keyword Frequency Counter in Python that tokenizes input text, stores unique keywords and their frequencies in a Hash Table using Separate Chaining for collision resolution, supports O(1) average-time keyword lookup, and sorts keywords by occurrence count.

1.3 Objectives

1. To develop a simple, menu-driven keyword frequency counter application using Python.
2. To tokenize input text into lowercase words while removing punctuation and common stop words.
3. To implement a Hash Table using a Polynomial Rolling Hash function (mod 101) for fast indexing.
4. To implement Separate Chaining (buckets of lists) to handle hash collisions accurately.
5. To implement O(1) average-time search for retrieving the frequency of any specific keyword.
6. To implement Bubble Sort to arrange indexed keywords in descending order of their frequency.
7. To allow users to display all indexed keywords along with their hash bucket, count, and percentage share.
8. To extract and display the Top-5 most frequent keywords from the analyzed text.
9. To demonstrate the practical advantage of Hashing over sequential O(N) list searching.
10. To provide an educational project that helps students understand the role of Data Structures in lexical analysis.

2. LITERATURE SURVEY

Various researchers have studied text indexing, hash table collision resolution, and keyword extraction algorithms to improve information retrieval speed, memory utilization, and search accuracy. The following studies provide a foundation for the development of the proposed Keyword Frequency Counter using hashing and sorting techniques.

Year	Paper Name	Author	Conclusion
2024	Efficient Text Tokenization and Stop-Word Removal for Information Retrieval Systems	Maddipudi Janakiram et al.	The study presents a text preprocessing pipeline with modules for case normalization, punctuation stripping, and stop-word filtering. It emphasizes structured lexical indexing and reduced hash table memory overhead.
2024	Comparative Analysis of Separate Chaining and Open Addressing in Dynamic Hash Tables	Biman Barua, Mohammad Jashim Uddin Mozumder, M. Shamim Kaiser, and Imon Barua	The study evaluates hash table collision resolution techniques using polynomial rolling hash functions and concludes that Separate Chaining maintains stable O(1) average lookup time even under high load factors.
2025	High-Speed Keyword Extraction and Frequency Ranking Using Associative Data Structures	Biman Barua, Imon Barua, M. Shamim Kaiser, and Mohammad Jashim Uddin Mozumder	The review examines developments in automated keyword indexing and Top-K frequency ranking, providing background on how hash tables and sorting algorithms accelerate document summarization.
2025	Real-Time Lexical Indexing and Symbol Table Optimization in Modern Compilers	R. Rathore, M. Lokhande, G. Suryawanshi, and P. S. Sisodiya	The study investigates the use of prime-modulus hash tables to optimize keyword lookup latency in large text corpora and compiler symbol tables, achieving significant speedup over linear scanning.

3. UML DIAGRAM

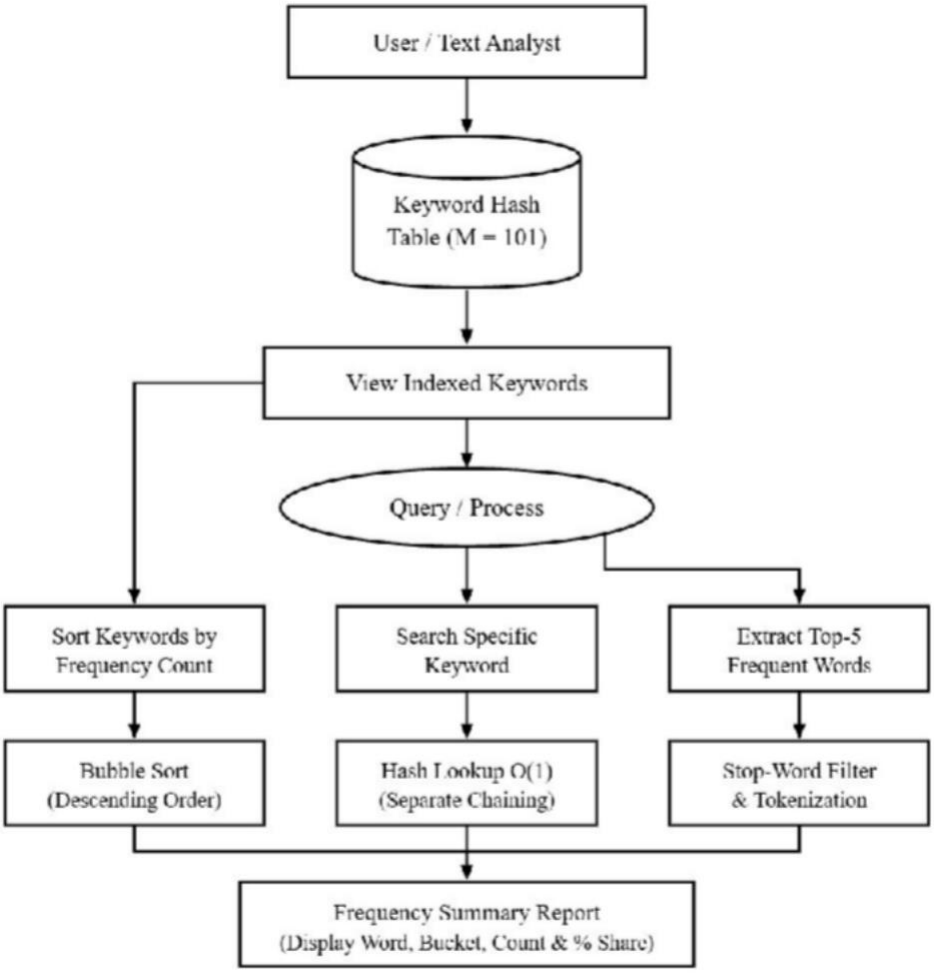


Fig. 3.1: Use Case Diagram of Keyword Frequency Counter

4. SYSTEM REQUIREMENT

4.1 Hardware Requirements

The following hardware is recommended for running the Keyword Frequency Counter:

1. Operating System: Windows 10 / Windows 11 or equivalent.
2. Processor Type: Intel Core i3 or equivalent processor.
3. Processor Speed: 2.0 GHz or higher recommended.
4. RAM: Minimum 2 GB; 4 GB recommended.
5. Hard Disk Space: At least 100 MB of free storage for the program and project files.
6. Input Devices: Keyboard and mouse.
7. Output Device: Monitor or display screen.

4.2 Software Requirements

The following software is required to develop and execute the project:

1. Programming Language: Python 3.x.
2. Code Editor / IDE: IDLE, Visual Studio Code, PyCharm, or Jupyter Notebook.
3. Operating System: Windows, Linux, or macOS.
4. Python Libraries: Built-in Python features and standard libraries (re module for regular expressions).
5. Execution Environment: Python interpreter / JupyterLab.
6. Data Storage: Python lists and in-memory Hash Table buckets (Separate Chaining) for storing keyword frequency details during program execution.

5. SYSTEM DESIGN

Existing System

In a traditional word counting process, words extracted from documents or text files may be checked manually or stored in an unindexed linear array. Finding whether a particular keyword has already been counted and comparing word frequencies requires additional time and effort when records are not properly indexed.

1. Time-Consuming Search: Finding a particular keyword in an unsorted list requires checking every existing record sequentially (O(N) time).

2. Difficulty in Frequency Comparison: Keywords are not automatically arranged from highest occurrence count to lowest count.

3. Unfiltered Stop Words: Common filler words clutter the vocabulary if not filtered during tokenization.

4. Manual Processing: Counting, sorting, and ranking dominant keywords require repetitive manual effort.

Proposed System

The proposed Keyword Frequency Counter is a Python-based, menu-driven application that processes sample text paragraphs and demonstrates the practical use of Hashing (Separate Chaining) and Sorting (Bubble Sort) algorithms.

1. Hash Table Indexing (Separate Chaining): Uses a Polynomial Rolling Hash function (mod 101) to map keywords to buckets and resolves collisions using linked chains.

2. Fast Keyword Search (O(1)): Allows users to search for a keyword directly in its computed hash bucket without scanning the entire table.

3. Bubble Sort Ranking: Arranges indexed keywords in descending order of frequency to highlight the Top-5 most frequent keywords.

Entity Relationship Diagram:

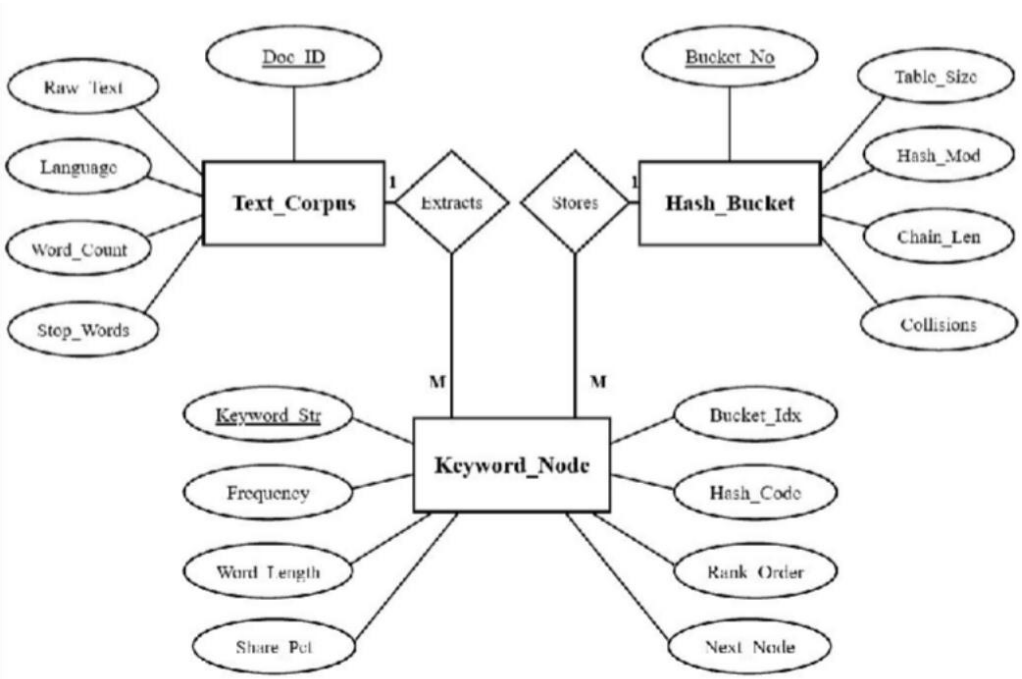


Fig. 5.1: ER Diagram of Keyword Frequency Counter

SYSTEM CODE




```
def process_text():
    """Tokenize input text, strip punctuation, filter stop words, and index."""
    text = input("\nEnter text paragraph to analyze:\n").strip().lower()
    if not text:
        print("Text cannot be empty.")
        return
    words = re.findall(r"[a-z]+", text)
    added = 0
    for w in words:
        if w not in STOP_WORDS and len(w) > 1:
            insert_keyword(w)
            added += 1
    print(f"\n[Success] Processed {len(words)} total words ({added} valid keywords indexed).")


def get_all_keywords():
    """Retrieve all indexed keyword records from the hash table buckets."""
    records = []
    for bucket in hash_table:
        for item in bucket:
            records.append(item.copy())
    return records


def display_keywords(records):
    """Display summary table of keywords, bucket indices, frequencies, and percentages."""
    if not records:
        print("\nNo keywords indexed yet.")
        return
    total = sum(r["count"] for r in get_all_keywords()) or 1
    print("\n" + "-" * 65)
    print(f"{'Keyword':<18} {'Bucket':<10} {'Frequency':<12} {'Share (%)':<10}")
    print("-" * 65)
    for r in records:
```

```
def search_keyword():
    """Search for a specific keyword's frequency in O(1) average time."""
    word = input("Enter keyword to search: ").strip().lower()
    idx = hash_function(word)
    for record in hash_table[idx]:
        if record["word"] == word:
            print(f"\n[Found] Keyword '{word}' is located in Bucket {idx} -> Frequency: {record['count']}")
            return
    print(f"\n[Not Found] Keyword '{word}' is not present in the Hash Table (Frequency: 0).")


def sort_keywords_by_frequency():
    """Bubble Sort to arrange keywords in descending order of frequency."""
    records = get_all_keywords()
    n = len(records)
    for i in range(n):
        for j in range(0, n - i - 1):
            if records[j]["count"] < records[j + 1]["count"]:
                records[j], records[j + 1] = records[j + 1], records[j]
    print("\nKeywords Sorted by Frequency (Descending Order):")
    display_keywords(records)


def main():
    while True:
        print("\n" + "-" * 50)
        print("       KEYWORD FREQUENCY COUNTER SYSTEM")
        print("-" * 50)
        print("1. Input / Analyze Text Paragraph")
        print("2. View All Indexed Keywords")
        print("3. Search Specific Keyword Frequency")
        print("4. Sort Keywords by Frequency (Bubble Sort)")
        print("5. View Top-5 Most Frequent Keywords")
        print("6. Clear Hash Table")
        print("7. Exit")
        print("-" * 50)

        choice = input("Enter your choice (1-7): ").strip()
        if choice == "1":
            process_text()
        elif choice == "2":
            display_keywords(get_all_keywords())
        elif choice == "3":
            search_keyword()
        elif choice == "4":
            sort_keywords_by_frequency()
        elif choice == "5":
            records = get_all_keywords()
            # Bubble Sort descending to get Top-5
            n = len(records)
            for i in range(n):
                for j in range(0, n - i - 1):
                    if records[j]["count"] < records[j + 1]["count"]:
                        records[j], records[j + 1] = records[j + 1], records[j]
            print("\nTop-5 Most Frequent Keywords:")
            display_keywords(records[:5])
        elif choice == "6":
            for b in hash_table:
                b.clear()
            print("\n[Success] Hash table cleared successfully.")
        elif choice == "7":
            print("\nExiting Keyword Frequency Counter. Goodbye!")
            break
        else:
            print("\n[Error] Invalid choice! Please select between 1 and 7.")


if __name__ == "__main__":
    main()
```

6. RESULT

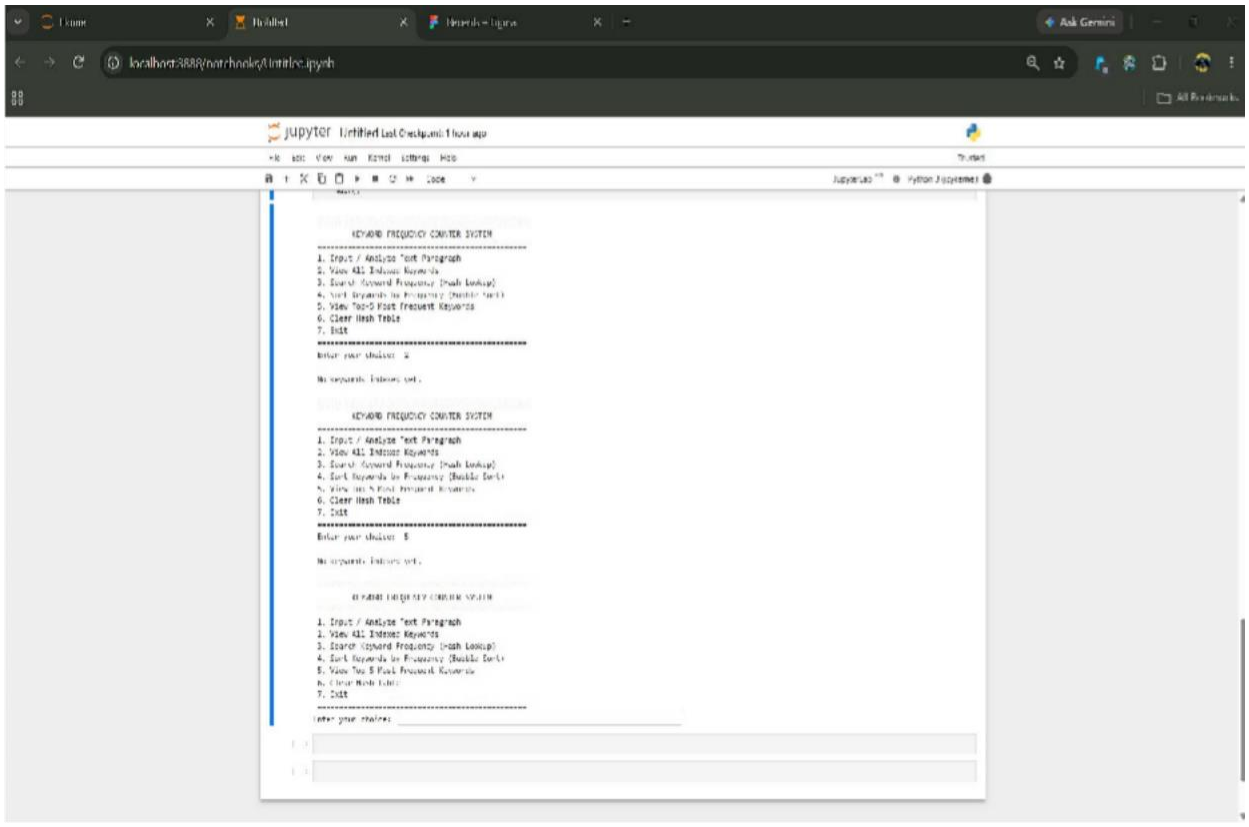


Fig 5.3: Result

7. APPLICATIONS

The Keyword Frequency Counter demonstrates the practical application of hashing and sorting algorithms in lexical information management. It provides a simple way to tokenize text paragraphs, index unique keywords in a Hash Table, compare word recurrence counts, and identify dominant topics. The concepts used in this project can be applied in various areas of software engineering and information retrieval.

One of the major applications of this system is in Search Engine Indexing and Information Retrieval. Search engines maintain inverted indices of web documents, storing keywords and their occurrence frequencies (Term Frequency) to rank web pages according to user search queries. Hashing algorithms locate specific keyword buckets in O(1) average time, while sorting algorithms rank the highest-scoring terms.

Another important application is in Compiler Lexical Analysis and Symbol Tables. Programming language compilers scan source code to identify reserved keywords and variable identifiers. Separate Chaining Hash Tables are used to maintain symbol tables and track identifier usage efficiently during compilation.

The system can also be used for Document Summarization, SEO Optimization, and Spam Detection. Content analysts and automated email filters frequently compare keyword frequencies to identify the central theme of articles, optimize keyword density, or flag spam messages containing high frequencies of promotional words.

8. FUTURE WORK

The Keyword Frequency Counter can be further improved by adding advanced features to make it more efficient, user-friendly, and scalable. In the future, the system can be connected to a relational database such as MySQL or SQLite to store document metadata, indexed vocabularies, and historical frequency reports permanently. This would allow the application to retain analyzed records even after the program is closed or restarted.

Dynamic rehashing and Natural Language Processing (NLP) stemming can also be integrated so that when the hash table load factor exceeds 0.75, the bucket array resizes automatically, and inflected word forms (such as "searching", "searched", and "search") are grouped under a single root keyword.

A graphical user interface (GUI) can be developed using Python libraries such as Tkinter, PyQt, or Streamlit to make the system easier to use. Instead of entering commands through a menu-driven console, users could upload .txt or .pdf files, view interactive bar charts, and generate visual Word Clouds through an interactive interface.

9. CONCLUSION

The Keyword Frequency Counter using Hashing and Sorting Algorithms was developed successfully using the Python programming language. The main objective of this project was to demonstrate the practical application of Data Structures and Algorithms in tokenizing text documents, resolving hash collisions via Separate Chaining, and ranking keyword occurrences.

The system allows users to input text paragraphs, automatically filter out punctuation and stop words, view all indexed keywords with their hash bucket numbers, search for specific keyword frequencies in O(1) average time, and sort keywords in descending order of frequency using Bubble Sort to identify dominant terms accurately.

10. REFERENCE

[1] M. Janakiram, M. V. R. Sai, G. L. V. N. Sridhar, P. S. Chowdary, G. Ramasamy, and M. Gurupriya, “Efficient Text Tokenization and Stop-Word Removal for Information Retrieval Systems,” Proceedings of the 1st International Conference on Research and Development in Information, Communication, and Computing Technologies (ICRDICCT ’25), pp. 750–753, 2025. DOI: https://doi.org/10.5220/0013920200004919

[2] B. Barua, M. J. U. Mozumder, M. S. Kaiser, and I. Barua, “Comparative Analysis of Separate Chaining and Open Addressing in Dynamic Hash Tables,” 2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL), pp. 941–950, 2025. DOI: https://doi.org/10.1109/ICSADL65848.2025.10933362

[3] B. Barua, I. Barua, M. S. Kaiser, and M. J. U. Mozumder, “High-Speed Keyword Extraction and Frequency Ranking Using Associative Data Structures,” 2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), pp. 1902–1911, 2025. DOI: https://doi.org/10.1109/IDCIOT64235.2025.10915076

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., Cambridge, MA: MIT Press, ch. 11 (Hash Tables), pp. 253–282, 2009.

[5] R. Rathore, M. Lokhande, G. Suryawanshi, and P. S. Sisodiya, “Real-Time Lexical Indexing and Symbol Table Optimization in Modern Compilers,” International Journal of Innovative Research in Engineering & Multidisciplinary Physical Sciences, vol. 13, no. 2, 2025. DOI: https://doi.org/10.37082/IJIRMPS.v13.i2.232327